

3주차 파이프라인을 어떻게 나누고, 무엇을 먼저 세울까

3주차 · 파이프라인 아키텍처

이 장의 수치는 모두 `practice/chapter3/` 의 코드를 실행해 얻은 값이다. 예시를 위해 지어낸 숫자는 없다. 인용한 값의 출처 파일은 출력 블록 첫 줄에 적었다.

- ☐ 1주차에서 데이터가 모델까지 가는 길을 보고 2주차에서 그 길에 흐르는 데이터가 어떻게 망가지는지 봤다면, 이 장은 그 길 자체를 설계함
- ☐ 무엇을 언제 계산할지, 파이프라인을 어떤 역할로 나눌지, 그 구성을 어떻게 파일 한 장으로 남길지를 정함
- ☐ 이 장은 설명과 실습을 나누지 않음
- ☐ 절마다 실행할 명령이 나오고 그 아래에 실제 출력과 해석이 이어짐
- ☐ 순서대로 실행하면 이 장의 모든 수치를 직접 생성함

1. 오늘의 질문

- ☐ 구성 파일 한 장으로 무엇을 검수할 수 있을까?
- ☐ 계층을 다섯으로 나눈다는 말은 서비스를 다섯 개 세운다는 뜻일까?
- ☐ 같은 집계를 두 벌 유지하면 무엇이 어긋나고, 그때 시스템은 멈출까?
- ☐ "그때 그 데이터로 그 계산을 다시 해 보라"는 요구에 답하려면 무엇이 있어야 할까?

2. 이 장에서 실행할 것

- ☐ 파이프라인 설계는 건물을 짓는 일과 한 가지가 같음
- ☐ 사람이 머릿속으로 아는 것을 도면에 적어 두어야 다른 사람이 같은 것을 다시 만들
- ☐ 도면 한 장에 자재·치수·순서를 적어 두면 다른 사람이 같은 건물을 세우듯, 구성 파일 한 장에 서비스·포트·의존을 적어 두면 다른 사람이 같은 환경을 세움
- ☐ 비유는 여기까지다
- ☐ 도면과 실물이 다르면 눈으로 보이지만, 구성 파일과 실제 동작이 어긋나면 실행해 보기 전에는 드러나지 않음
- ☐ 이 차이가 3절에서 처음 숫자로 나타남
- ☐ 파이프라인은 다섯 계층으로 나눔. 계층은 제품 이름의 서랍이 아니라 없을 때 무엇이 실패하는가로 정의됨

계층	없으면 실패하는 것	대표 도구
수집	유실·과부하	Kafka
저장	재처리·감사 불가	Postgres
처리	원천이 지표가 되지 못함	Spark, Airflow
서빙	데이터가 서비스되지 않음	FastAPI
모니터링	침묵 실패 미발견	Prometheus, Grafana

표 3.1 5계층과 없을 때 실패하는 것

- ☐ 실행하기 전에 먼저 적음. 가상 발주 조건은 넷임
- 민원 이벤트를 실시간으로 받아 저장하고 유입 현황을 대시보드로 봄
- 감사에 대비해 원천 이벤트를 일정 기간 되감아 다시 읽을 수 있어야 함
- 스트리밍 집계와 배치 정제는 이 단계에서 구현하지 않음(각각 5-6주차)
- 노트북 한 대에서 전부 돌아가야 함

표 3.2 구성 설계표 — `docker-compose.yml` 을 열기 전에 채운다

계층	넣는가(예/미름)	내가 고른 서비스	열 호스트 포트	무엇에 의존하는가	근거
수집					
저장					
처리					
서빙					
모니터링					

- ☐ 근거 칸에 "기본값이라서"를 쓰지 않음
- 발주 조건의 어느 문장에서 나왔는지, 또는 표 3.1의 "없으면 실패하는 것" 중 무엇인지를 가리킴
- ☐ 표를 다 채우면 **서비스 수·호스트 포트 수·의존성 수** 세 숫자를 예측값으로 적어 둠
- 남이 만든 파일을 먼저 열면 거기 적힌 목록이 정답처럼 보여 "왜 이 여섯 개인가"를 묻지 않게 됨
- ☐ 이 장에서 실행할 코드는 넷이며 순서가 정해져 있음

순서	실행할 파일	하는 일	절
1	3-1-compose-plan.py	compose 파일에서 서비스·포트·의존·보안 추출	3
2	3-2-layer-map.py	추출한 서비스를 5계층에 배정	4
3	3-3-lambda-skew.py	집계 로직 두 별의 수치가 어긋나는 상황 재현	5
4	3-4-replay-offset.py	오프셋을 되감아 같은 결과가 나오는지 확인	6

- 2번은 1번의 산출물을 입력으로 받으므로 **1번을 먼저 실행**해야 함
- 넷 다 Docker 없이 실행됨. 필요한 것은 Python과 PyYAML뿐임

```
cd practice/chapter3
pip install -r code/requirements.txt
```

3. 구성 파일 한 장이 설계도이자 검수 대상이다

- Docker는 애플리케이션과 실행에 필요한 파일·라이브러리·설정을 하나로 묶어 **컨테이너(container)** 라는 격리된 공간에서 돌리는 도구다
 - 여기서 격리는 컴퓨터 안에 또 다른 완전한 컴퓨터를 만든다는 뜻이 아님
 - 호스트의 운영체제 자원을 함께 쓰면서 프로세스·네트워크·파일 공간만 서로 구분한다는 뜻임
 - `docker run` 이 컨테이너 하나를 실행하는 명령이라면, `docker compose` 는 여러 컨테이너와 그 연결 관계를 한 번에 관리하는 명령이다
- Compose 파일은 **선언적 구성**이다
 - 명령을 순서대로 입력하는 절차적 방식과 달리 "무엇이 있어야 하는가"의 최종 상태를 적음
 - 그래서 구성 자체가 사람이 읽는 문서이면서 실행 가능한 설계도가 됨
 - 이 성질이 공공 검수에서 중요함 — 발주자가 같은 파일로 같은 구성을 다시 세워 확인할 수 있기 때문임
- `docker-compose.yml` 은 여러 컨테이너의 구성을 적는 YAML 파일임
 - YAML은 들여쓰기로 항목의 관계를 표현하는 텍스트 형식임
 - 표와 비슷하게 이름: 값 을 쓰지만, 들여쓰기 깊이가 부모·자식 관계를 결정함
 - 따라서 탭과 공백을 섞지 않고, 같은 단계의 항목을 같은 칸에 맞춰야 함
 - 다음은 3장 실습 파일에서 핵심 부분만 줄인 예시임

```

services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.5.0
    ports:
      - "2181:2181"

  kafka:
    image: confluentinc/cp-kafka:7.5.0
    depends_on:
      - zookeeper
    ports:
      - "9092:9092"

```

- `services` 는 실행할 서비스 목록의 시작점임
- `zookeeper` 와 `kafka` 는 각각 실행할 컨테이너의 이름임
- `image` 는 컨테이너를 만들 때 사용할 이미지와 버전임
 - 여기서 이미지는 사진·그림 파일이 아니라, 프로그램·라이브러리·기본 설정·파일 시스템을 묶어 둔 실행 재료임
 - 이미지는 여러 컨테이너가 함께 사용할 수 있는 읽기 전용 원본이고, 컨테이너는 그 원본을 실제로 실행한 인스턴스임
 - `confluentinc/cp-zookeeper:7.5.0` 에서 `cp-zookeeper` 는 이미지 이름이고 `7.5.0` 은 사용할 버전 태그임
- `venv` 와 컨테이너는 격리하는 대상이 다름
- `venv` 는 한 컴퓨터 안에서 Python 패키지를 프로젝트별로 나누는 기능임
 - 3장의 네 Python 실습은 Python과 PyYAML만 필요하므로 `venv` 로 실행함
- 컨테이너는 Python 패키지뿐 아니라 Kafka·Zookeeper·PostgreSQL 같은 별도 서버와 네트워크·포트·실행 설정까지 함께 나눔
 - `venv` 에 Kafka를 설치해도 Kafka 서버와 Zookeeper의 실행 환경까지 자동으로 재현되지는 않음
- 따라서 두 도구를 함께 쓰는 경우가 많음
 - Docker Compose가 Kafka와 PostgreSQL을 실행하고, 각 Python 애플리케이션은 자기 `venv` 에서 실행할 수 있음
- 컨테이너가 항상 더 좋은 선택은 아님
 - Python 코드만 시험하거나 가벼운 분석을 할 때는 `venv` 가 설치와 실행이 단순함
 - 여러 서버를 같은 버전과 연결 구조로 재현해야 할 때 컨테이너가 적합함
- `ports` 의 `2181:2181` 은 호스트 포트:컨테이너 포트 라는 뜻임
 - 컴퓨터의 2181번 포트로 들어온 요청을 Zookeeper 컨테이너의 2181번 포트로 전달함
- `depends_on` 은 Kafka를 시작하기 전에 Zookeeper를 먼저 시작해야 한다는 관계를 적음
 - 단순한 `depends_on` 만으로 Zookeeper가 실제 요청을 받을 준비까지 끝났다고 보장하지는 않음
 - 실습 파일은 `healthcheck` 와 시작 조건을 함께 써서 준비 상태를 확인함
- 이 파일에는 "1번 명령 실행 후 2번 명령 실행"을 적지 않음

- 필요한 서비스·버전·포트·의존 관계를 적고, `docker compose up` 이 그 상태를 만들

□ 선언적 구성에는 함정이 하나 붙음

○ 구성이 문서가 된다는 말은 그 파일이 저장소에 커밋되어 여러 사람이 본다는 뜻이기도 함

○ 그래서 비밀번호와 인증키는 코드나 구성 파일이 아니라 환경변수·시크릿 저장소에 두고, 파일에는 참조만 남김

□ 첫 번째 코드를 실행함. 이 절의 숫자는 전부 이 명령의 출력임

```
cd practice\chapter3
python code\3-1-compose-plan.py # 윈도우
python3 code/3-1-compose-plan.py # 맥, 리눅스
```

출처: practice/chapter3/results/3-1-compose-plan.log

[2] 서비스 6개

zookeeper	confluentinc/cp-zookeeper:7.5.0	ports=2181:2181
kafka	confluentinc/cp-kafka:7.5.0	ports=9092:9092, 29092:29092
postgres	postgres:15	ports=5432:5432
prometheus	prom/prometheus:v2.47.0	ports=9090:9090
grafana	grafana/grafana:10.1.0	ports=3000:3000
kafka-ui	provectuslabs/kafka-ui:latest	ports=8080:8080

[3] 호스트 포트 7개

2181 3000 5432 8080 9090 9092 29092

서비스 하나가 포트 둘을 엮: kafka → 9092:9092, 29092:29092

[4] 의존성 - 추출 2건 / 원본 3건

추출됨 : grafana → prometheus

추출됨 : kafka-ui → kafka

누락됨 : kafka (depends_on이 맵 형태여서 이 파서가 읽지 않음)

[5] 보안 점검 - 경고 2건

postgres: weak/default password in POSTGRES_PASSWORD

grafana: weak/default password in GF_SECURITY_ADMIN_PASSWORD

□ 화면에 나온 값에서 읽을 것

○ **서비스 6개** — 표 3.2에서 계층을 셋만 넣기로 했다면 예측은 3이었을 것임. 계층 하나에 서비스 하나가 대응하지 않는다는 사실이 여기서 처음 드러나며, 4절에서 계층별로 세어 봄

○ **포트가 서비스보다 하나 많음** — kafka가 9092와 29092 둘을 엮. 9092는 호스트에서 접속하는 통로 (PLAINTEXT_HOST)이고 29092는 컨테이너끼리 쓰는 통로 (PLAINTEXT)임. "서비스 하나에 포트 하나"라는 예측은 여기서 깨짐

○ 의존성 추출 2건, 원본 3건 — compose 파일에는 kafka → zookeeper , grafana → prometheus , kafka-ui → kafka 셋이 적혀 있음. 스크립트가 리스트 형태로 적힌 depends_on 만 읽고, kafka가 헬스체크 조건(맵 형태)으로 적은 의존은 지나침. **어긋난 것은 설계가 아니라 도구임**

○ 경고 2건 — POSTGRES_PASSWORD 의 값은 secure_password 인데도 경고가 뜸. 경고가 가리키는 것은 값의 복잡도가 아니라 **평문이 파일에 존재한다는 사실 자체임**

□ 예측한 세 숫자 옆에 실제 값을 적고, 어긋난 항목의 원인을 규명함

○ 예측과 산출물이 어긋났을 때 자기 판단을 고칠지 도구를 고칠지는 원본과 대조해야 정해짐

○ 설계를 먼저 적지 않으면 이 질문 자체가 생기지 않고 "의존성 2개"를 그대로 받아 적게 됨

- 자동 추출 산출물만 근거로 보고하면 그 보고가 틀린다
- 이 실행은 의존성을 2건으로 냈지만 원본에는 3건이 있다. 파서가 읽는 형식이 한 가지뿐이기 때문이다.
- 검수 문서에 "의존성 2건"이라고 적기 전에 원본 파일을 열어 대조한다.

4. 계층은 도구 목록이 아니라 없을 때 실패하는 것이다

□ 파이프라인을 다섯 계층으로 나눌 때 흔한 오해는 계층을 제품 이름의 서랍으로 보는 것임

○ 계층은 역할 분해이고, 그 역할은 표 3.1처럼 **그 계층이 없을 때 무엇이 실패하는가**로 정의됨

□ 수집 계층은 생산과 소비의 속도 차이를 흡수함

○ 재난 문자 발송 직후 민원이 초당 수백 건 들어올 때 하류 서버는 그 속도를 못 따라감

○ 완충 로그를 두면 빠르게 받고 천천히 처리할 수 있음

○ 이 계층의 핵심 판단은 전달 보증 수준의 선택임 — 4주차에서 커밋 시점 한 줄만 바꾸면 유실 20건과 중복 10건이 갈림(practice/chapter4/data/output/ch4_delivery_report.json)

□ 저장 계층은 원천과 가공 데이터를 보존하고 접근을 통제함

○ 원천을 버리면 세 가지가 불가능해짐 — 변환 로직 버그를 나중에 발견했을 때의 재처리, "그때 받은 그대로"를 요구받는 감사, 처음에는 안 쓰던 필드가 나중에 필요해지는 새 요구

○ 셋 다 요약본으로는 복원되지 않음

□ 처리 계층은 수집된 데이터를 변환·집계함

○ 원천 이벤트는 그대로는 지표가 되지 못하므로 지역명을 표준 코드로 바꾸고 시간 창으로 묶어 썸

○ 늦게 도착한 이벤트를 얼마나 기다렸다 창을 닫을지 정하는 것이 2주차에서 본 **워터마크(watermark)**이며, 5절에서 그 값을 바꿔 봄

□ 서빙 계층은 조회·리포팅·예측 요구에 통제된 성능으로 데이터를 내줌

○ 운영에서의 질문은 "새 모델을 어떻게 넣는가"보다 **"잘못됐을 때 어떻게 되돌리는가"** 다

□ 모니터링 계층이 없으면 침묵 실패가 발견되지 않음

○ 무엇을 볼지의 출발점은 Google SRE의 골든 시그널 네 가지, 즉 지연·트래픽·오류·포화임

○ 11주차는 PSI 0.3324가 경보 대역인데 KS 검정은 유의하지 않은 상태를 실측해 단일 지표로 판정하면 안 되는 이유를 보임(practice/chapter11/data/output/ch11_drift_report.json)

□ 두 번째 코드를 실행함. 3절이 추출한 서비스 여섯을 계층에 배정함

```
python3 code/3-2-layer-map.py
```

```
# 출처: practice/chapter3/results/3-2-layer-map.log
```

[1] 계층 배정 - 서비스 6개를 계층 5개에 배정

수집	zookeeper, kafka
저장	postgres
처리	(없음)
서빙	(없음)
모니터링	prometheus, grafana, kafka-ui

[2] 계층당 서비스 수와 포트 수

계층	서비스	포트	없으면 실패하는 것
수집	2	3	유실·과부하
저장	1	1	재처리·감사 불가
처리	0	0	원천이 지표가 되지 못함
서빙	0	0	데이터가 서비스되지 않음
모니터링	3	3	침묵 실패 미발견
합계	6	7	

계층 5개 대 서비스 6개 - 1대 1이 아님

[3] 비어 있는 계층 2개

처리	채우는 시점: 5주차(스트리밍)·6주차(배치)
서빙	채우는 시점: 10주차(모델 배포)

[4] 배정 규칙을 바꾸면 - kafka-ui를 모니터링이 아니라 수집으로 보면

수집	2개 → 3개
모니터링	3개 → 2개

□ 화면에 나온 값에서 읽을 것

○ **계층과 서비스는 1대 1이 아님** — 수집 하나를 세우는 데 서비스 둘(kafka와 그 메타데이터를 관리하는 zookeeper)이 들고, 모니터링 하나에 셋이 듬. 표 3.2에서 서비스 수를 계층 수로 예측했다면 여기서 어긋남

○ **빈 계층이 둘** — 처리와 서빙 자리가 비어 있음. 생략이 아니라 범위 조정이며, 이 장에서 보이려는 것이 계층이 나뉘어 있다는 사실이므로 수집·저장·모니터링만으로 관찰됨. **실습이 막히는 시점이 그 계층을 더할 시점이다**

○ **배정 규칙을 바꾸면 개수가 바뀐다** — kafka-ui는 Kafka의 토픽과 메시지를 화면으로 보는 도구여서 모니터링으로도, 수집 계층의 운영 도구로도 볼 수 있음. 서비스 목록은 그대로인데 배정 규칙만 바뀌도

계층당 개수가 달라짐

○ 그러므로 "모니터링 계층에 서비스가 셋"이라는 문장은 자료가 정한 값이 아니라 **배정 규칙이 정한 값**임.
규칙을 함께 밝히지 않으면 검증할 수 없음

5. 재처리를 두 벌 로직으로 감당하면 — Lambda

□ 파이프라인 설계에서 가장 먼저 정하는 것은 "데이터를 언제 계산하는가"임

○ **배치(batch)** 는 데이터를 일정 기간 모았다가 한꺼번에 처리함. 하루분 전체를 대상으로 계산하므로 늦게 도착한 이벤트까지 포함한 확정 수치를 낼 수 있고, 같은 입력으로 다시 돌리면 같은 결과가 나옴. 대신 지연이 큼

○ **스트리밍(streaming)** 은 이벤트가 도착하는 즉시 처리함. 재난 신고가 물리는 순간의 건수를 초 단위로 갱신할 수 있음. 대신 집계가 어려움 — 23시 58분에 발생한 민원이 00시 03분에 도착하면 이미 닫힌 집계에 넣을지를 매번 결정해야 함

□ 공공 업무는 둘을 동시에 요구함

○ 재난 상황실은 지금의 신고 건수를 봐야 하고, 사후 감사와 통계 보고는 그날 확정된 정확한 건수를 요구함

○ 하나의 처리 방식으로는 두 요구를 함께 만족시킬 수 없다는 것이 **하이브리드(hybrid)** 의 출발점이며, 대표 설계가 Lambda와 Kappa다

□ Lambda는 같은 데이터를 두 경로로 동시에 흘려보냄

○ 배치 레이어는 원천 전체를 주기적으로 재계산해 확정치를 만들고, 스피드 레이어는 방금 도착한 데이터만 처리해 근사치를 만듦

○ 조회할 때 둘을 합쳐 "확정된 과거 + 근사한 최근"을 보여 줌

□ 정확성에는 대가가 따름

○ **같은 집계 로직을 배치용과 스트리밍용 두 벌로 유지해야 한다.**

○ 결측 처리 규칙이나 창 경계 같은 기준 하나를 한쪽만 고치면 같은 시각의 확정치와 근사치가 어긋남

○ 그런데 시스템은 멈추지 않음. 1주치의 침묵 실패가 아키텍처 수준에서 다시 나타나는 자리이며, 아래에서 이것을 재현함

□ 세 번째 코드를 실행함. 서울 대기질 측정소 자료 두 시점(22시·23시, 각 40건)에 같은 집계를 두 벌 적용함

```
python3 code/3-3-lambda-skew.py
```

출처: practice/chapter3/results/3-3-lambda-skew.log

[1] 입력 - 이벤트 80건, 측정소 40개

창 길이 60분, 창 2개

도착 시각은 구성값 - 기본 3분, 5번째마다 72분

자연 도착으로 구성한 이벤트 16건

[2] 두 로직의 자연 허용이 같을 때 (배치 전체 / 스피드 15분)

창	배치 건수	스피드 건수	배치 평균	스피드 평균
2026-06-28 22:00	40	40	20.02	20.02
2026-06-28 23:00	40	40	20.77	20.77

→ 어긋난 창 0개

[3] 스피드 레이어만 자연 허용을 15분 → 10분으로 줄인 뒤

창	배치 건수	스피드 건수	배치 평균	스피드 평균
2026-06-28 22:00	40	32	20.02	19.66
2026-06-28 23:00	40	32	20.77	20.84

→ 어긋난 창 2개

[4] 이 실행이 끝나는 방식

2026-06-28 22:00 - 스피드가 8건을 놓쳐 평균이 20.02 대 19.66로 갈림

2026-06-28 23:00 - 스피드가 8건을 놓쳐 평균이 20.77 대 20.84로 갈림

예외 없음, 종료 코드 0 - 두 수치가 어긋난 채로 정상 종료함

□ 화면에 나온 값에서 읽을 것

○ **규칙이 같으면 두 수치도 같음** — 자연 허용이 양쪽 다 15분일 때 어긋난 창은 0개임. 로직이 두 벌이라는 사실 자체가 문제를 만드는 것이 아님

○ **한쪽만 5분 줄였더니 창마다 8건이 빠짐** — 운영 부담을 이유로 스피드 레이어의 자연 허용만 10분으로 줄이면, 창 마감 뒤에 도착한 측정소 자료가 근사치에서 사라짐

○ **오차의 방향이 일정하지 않음** — 22시 창은 평균이 20.02에서 19.66으로 내려가고 23시 창은 20.77에서 20.84로 올라감. 누락이 한 방향으로만 치우치지 않으므로 "근사치는 대체로 낮게 나온다" 같은 보정이 성립하지 않음

○ **예외 없이 종료 코드 0** — 두 수치가 어긋난 채로 프로그램은 정상 종료함. 로그에 오류가 남지 않고, 대시보드는 계속 숫자를 보여 줌

- **도착 시각은 구성값이다**

- 원자료에는 측정값과 측정 시각만 있고 "언제 도착했는가"는 없다. 자연 도착을 재현하려고 도착 시각을 코드에 고정된 규칙으로 만들었다(`ARRIVAL_RULE`).

- 난수를 쓰지 않으므로 몇 번 실행해도 같은 값이 나오지만, 이 16건의 지연은 실제로 관측된 지연이 아니다. 측정값 자체는 공공 API 응답 그대로다.

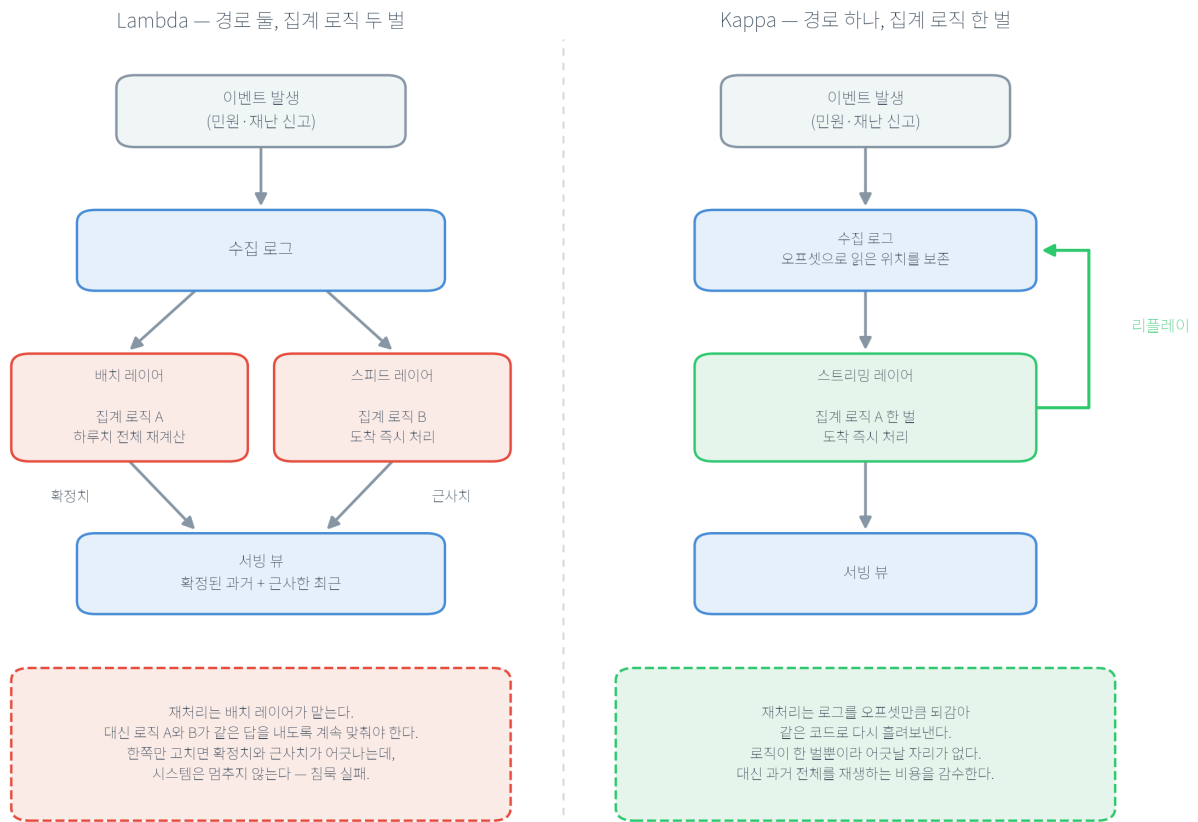


그림 3.1 Lambda와 Kappa: 재처리를 어디서 감당하는가. 위쪽 절반은 같고 — 같은 이벤트가 같은 수집 로그로 들어감 — 그 아래에서 갈라진다. Lambda는 경로가 둘이고 Kappa는 하나이며, 오른쪽 초록 화살표가 로그를 되감아 같은 코드로 다시 흘리는 리플레이다.

6. 재처리를 리플레이로 감당하면 — Kappa와 감사 대응

- ☐ Kappa는 배치 레이어를 두지 않고 스트리밍 하나만 유지함
- ☐ 재처리가 필요하면 로그에 보관된 과거 이벤트를 처음부터 다시 흘러보내는데, 이것을 **리플레이(replay)**라 한다
- ☐ 로직이 한 벌뿐이므로 5절에서 본 어긋남이 구조적으로 사라짐
- ☐ 성립하는 전제는 이벤트 로그를 보존하고 다시 읽을 수 있어야 한다는 것임
- ☐ Kappa 아키텍처는 Kafka 같은 로그 기반 수집 계층을 사용해 이벤트를 일정 기간 보관하고, 오프셋을 되감아 과거 데이터를 다시 처리한다.
- ☐ 그러므로 **컨슈머(consumer, Kafka 로그를 읽는 처리 프로그램)**를 과거 오프셋으로 되돌리면 그 시점부터 다시 읽을 수 있음
- ☐ 공공에서 이 능력은 감사 대응 능력이 됨

- 감사관이 "왜 이 통계가 이렇게 나왔는가"를 물으면, 답은 결국 "그 시점에 들어온 그 데이터로 그 계산을 다시 만들어낼 수 있는가" 로 좁혀짐
- 로그를 보존하고 오프셋으로 되감을 수 있는 아키텍처는 재현해 보이겠다고 답할 수 있고, 요약본만 남긴 아키텍처는 답할 수 없음
- 그래서 공공 설계의 원칙은 하나임 — 증거를 사람이 사후에 쓰는 문서가 아니라 시스템이 실행 중에 남기는 산출물로 둔다
- 한계도 분명함
- 대규모 과거 데이터 전체를 다시 흘리는 것은 최적화된 배치 스캔보다 비효율적일 수 있고, 복잡한 다차원 집계나 대규모 조인은 여전히 배치 엔진이 유리함
- 네 번째 코드를 실행함. 5절과 같은 자료에 오프셋을 붙여 로그로 만든 뒤 되감아 읽음

```
python3 code/3-4-replay-offset.py
```

```
# 출처: practice/chapter3/results/3-4-replay-offset.log
```

```
[1] 이벤트 로그 - 80건, 오프셋 0~79
```

```
보존 기간이 지난 구간: 오프셋 0~39 (40건)
```

```
되감을 수 있는 구간: 오프셋 40~79
```

```
[2] 최초 집계 (대표값 mean)
```

```
2026-06-28 23:00 건수 40 대표값(mean) 20.77 최소 14 최대 27
```

```
결과 해시 f6a5c48ba48fa6ef...
```

```
[3] 오프셋을 되감아 같은 코드로 다시 읽기
```

```
2026-06-28 23:00 건수 40 대표값(mean) 20.77 최소 14 최대 27
```

```
결과 해시 f6a5c48ba48fa6ef...
```

```
해시 일치: 예 - 그 시점 자료로 그 계산을 다시 만들었음
```

```
[4] 집계 규칙을 고친 뒤 되감기 (대표값 mean → median)
```

```
2026-06-28 23:00 건수 40 대표값(median) 20.0 최소 14 최대 27
```

```
결과 해시 5cfafcb513d8cdf9...
```

```
해시 변경: 예 - 로직 한 벌만 고쳐 확정치를 다시 냈음
```

```
[5] 보존 기간 밖을 요청하면
```

```
오프셋 0부터 요청했으나 40건은 남아 있지 않음
```

```
실제로 읽은 건수 40건 - 요청한 80건에 못 미침
```

```
감사 질의가 이 구간을 향하면 재현해 보일 수 없음
```

□ 화면에 나온 값에서 읽을 것

○ 해시가 같으면 재현된 것임 — 되감아 다시 읽은 결과의 지문이 처음과 같은 `f6a5c48b...` 임. "다시 해봤더니 비슷했다"가 아니라 같은 값을 한 줄로 보일 수 있음

○ 로직을 고치면 해시가 바뀜 — 대표값을 평균에서 중앙값으로 바꾸자 20.77이 20.0이 되고 지문도 달라짐. Lambda라면 이 수정을 배치용과 스트리밍용 두 곳에 해야 하지만, 여기서는 한 번만 고쳤음

○ 보존 기간이 재현의 한계선임 — 오프셋 0부터 요청했지만 앞의 40건은 남아 있지 않아 22시 창은 재현되지 않음. 아키텍처가 리플레이를 지원해도 보존 기간 밖의 질의에는 답할 수 없음

○ 그러므로 "우리는 재현할 수 있습니다"라는 답에는 **기간이 붙어야 함** — 며칠까지 되감을 수 있는지가 곧 감사 대응 범위임

- 평균과 중앙값의 차이 0.77은 이 자료에서만 값이다
- 여기서 확인한 것은 두 대표값의 크기 차이가 아니라, 규칙을 바꾼 뒤 리플레이로 새 확정치를 낼 수 있다는 사실이다.
- 어느 대표값이 옳은지는 지표의 목적이 정한다. 측정소 하나의 튀는 값에 흔들리지 않아야 하면 중앙값을 쓴다.

7. SHA-256은 데이터의 지문이다

□ SHA-256은 파일이나 데이터의 내용을 일정한 형식의 해시값으로 바꾸는 계산 방법임

○ 같은 파일을 다시 계산하면 같은 해시값이 나옴

○ 파일 내용이 한 글자만 바뀌어도 해시값은 크게 달라짐

○ 결과는 보통 64자리 16진수 문자열로 표시함

□ SHA-256은 암호화와 다름

○ 암호화는 정해진 키로 원래 내용을 다시 복원할 수 있음

○ 해시는 원래 파일을 복원하는 용도가 아니라 두 데이터가 같은지 비교하는 용도임

□ 6절의 리플레이 실습에서는 집계 결과의 SHA-256 해시를 비교함

○ 처음 계산한 결과와 로그를 되감아 다시 계산한 결과의 해시가 같으면 같은 입력과 규칙으로 같은 결과를 만들었다고 판단함

○ 평균을 중앙값으로 바꾼 뒤 해시가 달라진 것은 계산 규칙이 바뀌었다는 뜻임

○ 따라서 해시는 "결과가 비슷하다"가 아니라 "결과가 같은가"를 확인하는 기준임

8. 핵심 요약

□ 1. 구성 파일은 설계도이자 검수 대상이며, 자동 추출 산출물은 원본과 대조해야 한다. (3절)

○ compose 파일 한 장에서 서비스 6개·호스트 포트 7개·보안 경고 2건이 나왔고, `POSTGRES_PASSWORD`의 값이 `secure_password` 인데도 경고가 뜬 것은 평문이 파일에 있다는 사실 자체를 가리키기 때문임

○ 같은 실행이 의존성을 2건으로 냈지만 원본에는 3건이 있음. 검수 문서에 옮기기 전에 원본을 열어

대조함

☐ 2. 계층은 도구 목록이 아니라 없을 때 실패하는 것의 목록이며, 서비스와 1대 1이 아니다. (4절)

○ 서비스 6개가 계층 5개에 배정되어 수집 2·저장 1·처리 0·서빙 0·모니터링 3이 됨

○ kafka-ui를 어느 계층으로 보느냐에 따라 개수가 바뀌므로, 계층별 수를 보고할 때는 배정 규칙을 함께 밝힘

☐ 3. Lambda의 비용은 로직 두 벌이고, 어긋남은 오류 없이 지나간다. (5절)

○ 지연 허용을 한쪽만 15분에서 10분으로 줄이자 창마다 8건이 빠지고 평균이 20.02 대 19.66, 20.77 대 20.84로 같았음

○ 오차의 방향이 창마다 달랐고, 프로그램은 예외 없이 종료 코드 0으로 끝났음

☐ 4. Kappa의 리플레이는 로직 한 벌로 재처리하는 방법이고, 재현은 해시로 보인다. (6절)

○ 오프셋을 되감아 다시 읽은 결과의 지문이 처음과 같았고, 대표값 규칙을 고친 뒤에는 한 벌만 수정해 새 확정치를 냈음

○ 선택의 축은 정확성이 아니라 재처리를 로직 이중화로 감당할지 리플레이 비용으로 감당할지다

☐ 5. 공공에서 리플레이는 감사 대응 능력이며, 그 범위는 보존 기간이 정한다. (6절)

○ 오프셋 0부터 요청했으나 앞의 40건이 남아 있지 않아 그 구간은 재현되지 않았음

○ 증거는 사람이 사후에 쓰는 문서가 아니라 시스템이 실행 중에 남기는 산출물이어야 하고, "재현할 수 있다"는 답에는 며칠까지인지가 붙어야 함

9. 과제

☐ 실습을 순서대로 실행해 만든 산출물 네 개를 LMS에 제출한다

1. 표 3.2 구성 설계표 — 실행 결과와 대조해 고친 최종본. 텍스트든 이미지든 형식은 무방하다.

2. `practice/chapter3/data/output/ch3_compose_plan.json`

3. `practice/chapter3/data/output/ch3_layer_map.json`

4. `practice/chapter3/data/output/ch3_lambda_skew.json`

☐ 이어서 네 가지를 각각 세 문장 이내로 쓴다

1. 본인 산출물의 `compose_path`와 `generated_at`을 그대로 옮겨 적고, `compose_path`가 학생마다 다른 이유를 쓴다.

2. 예측한 서비스 수·포트 수·의존성 수와 실제 값을 나란히 적고, 어긋난 항목마다 자기 판단을 고쳤는지 도구의 한계였는지 밝힌다.

3. `ch3_layer_map.json`에서 비어 있는 계층 둘을 적고, 그 계층을 언제 추가해야 하는지 근거와 함께 쓴다.

4. `ch3_lambda_skew.json`의 `mismatch_after`에 적힌 창과 누락 건수를 옮겨 적고, 이 실행의 종료 코드가 0인 것이 왜 문제인지 쓴다.

☐ 값을 지어내지 않는다

☐ 실행이 안 되면 어디서 막혔는지 그대로 쓴다

☐ 수업일 포함 7일 이내에 제출한다

10. 더 알아보기

- ☐ 실습을 더 해 보려면 `practice/chapter3/` 에서 다음을 시도함
- ☐ `3-1-compose-plan.py` 의 파서를 고쳐 맵 형태 `depends_on` 까지 읽게 만들고, 의존성이 3건으로 바뀌는지 확인함
- ☐ `3-2-layer-map.py` 의 `LAYER_OF` 표를 고쳐 배정 규칙이 계층별 개수를 어떻게 바꾸는지 확인함
- ☐ `3-3-lambda-skew.py` 의 `GRACE_AFTER` 를 여러 값으로 바꿔, 지연 허용을 몇 분까지 줄여야 어긋남이 시작되는지 찾음
- ☐ `docker-compose.yml` 에 서비스를 하나 더 추가하거나 이미 쓰는 포트를 중복 지정해 무엇이 달라지는지 봄
- ☐ 뒤 주차에서 이어지는 내용
- ☐ 수집 계층의 전달 보증과 중복 제거는 4주차, 워터마크와 창 집계는 5주차, 배치 정제와 재현은 6주차에서 직접 실행함
- ☐ 비어 있던 서빙 계층은 10주차에, 모니터링 지표 판정은 11주차에 채움
- ☐ 원문을 직접 보고 싶다면 다음 두 편이 출발점임
- ☐ Pathirage, 「Kappa Architecture」 — Kappa의 정의와 리플레이 전제를 정리한 문서. <https://milinda.pathirage.org/kappa-architecture.com/>
- ☐ The Twelve-Factor App, 「III. Config」 — 비밀과 구성을 환경에 두는 원칙. <https://12factor.net/config>